String Manipulation and Regular Expressions in R (2)

Chapter 4

Guani Wu

The Caret Metacharacter Revisited

Question: What is the difference between `^[0-9]`, `[^0-9]`, and `[0-9^]`?

Quantifiers

Quantifiers can be attached to literal characters, character classes, or groups to match repeats.

Pattern	Meaning
`*`	Match 0 or more (is greedy)
`+`	Match 1 or more (is greedy)
`?`	Match 0 or 1
`{3}`	Match Exactly 3
`{3,}`	Match 3 or more
`{3,5}`	Match 3, 4 or 5

Quantifier Examples

```
text <- "words or numbers
9,876 and combos12
like password_1234"

str_view(text, "\\S")
```

```
## [1] | <w><o><r><d><s> <o><r> <n><u><m><b><e><r><s>
##     | <9><,><8><7><6> <a><n><d> <c><o><m><b><o><s><1><2>
##     | <l><i><k><e> <p><a><s><s><w><o><r><d><_><1><2><3><
```

Quantifier Examples

```
text <- "words or numbers 9,876
and combos12
like password_123"

str_view(text, "\\S+")
```

```
## [1] | <words> <or> <numbers> <9,876>
##     | <and> <combos12>
##     | <like> <password_123>
```

Quantifier Examples

```
text <- "words or numbers 9,876
and combos12
like password_123"

str_view(text, "\\d")
```

```
## [1] | words or numbers <9>,<8><7><6>
##     | and combos<1><2>
##     | like password_<1><2><3>
```

Quantifier Examples

```
text <- "words or numbers 9,876
and combos12
like password_123"

str_view(text, "\\d+")
```

```
## [1] | words or numbers <9>,<876>
##     | and combos<12>
##     | like password_<123>
```

Quantifier Examples

```
text <- "the year 1996 area code 310 combo123
password_1234 singledigit 5"
str_replace_all(text, "\\d{3}", "-")
```

```
## [1] "the year -6 area code - combo- \npassword_-4 single
```

```
str_replace_all(text, "\\d{2,4}", "-")
```

```
## [1] "the year - area code - combo- \npassword_- singledi
```

Quantifier Examples

```
text <- "the year 1996 area code 310 combo123
password_1234 singledigit 5"
str_replace_all(text, "\\d?", "-")
```

```
## [1] "-t-h-e- -y-e-a-r- ----- -a-r-e-a- -c-o-d-e- ----
-c-o-m-b-o---- -\n-p-a-s-s-w-o-r-d-_----- -s-i-n-g-l-e-d-i-
```

Greedy vs Ungreedy Matching

Quantifiers are by default **greedy** in the sense that they will return the longest match.

Adding `?` to a quantifier will make it ungreedy (or **lazy**).

```
text <- "Peter Piper picked a peck of pickled peppers"
str_extract(text,"P.*r")
```

```
## [1] "Peter Piper picked a peck of pickled pepper"
```

```
str_extract(text,"P.*?r")
```

```
## [1] "Peter"
```

Greedy vs Ungreedy Matching

```
text <- "Peter Piper picked a peck of pickled peppers"
str_extract_all(text, "P.*?r")
```

```
## [[1]]
## [1] "Peter" "Piper"
```

```
str_extract_all(text,"[Pp].*?r")
```

```
## [[1]]
## [1] "Peter"
## [2] "Piper"
## [3] "picked a peck of pickled pepper"
```

Grouping and Capturing

Parentheses `(` `)` define a **group** that groups together parts of a regular expression.

Besides grouping part of a regular expression together, parentheses also create a numbered **capturing group**: Any matches to the part of the pattern defined by the parentheses can be referenced by group number, either for modification or replacement.

By including `?:` after the opening parenthesis, the group becomes a **non-capturing group**.

For example, in the pattern `(abc)(def)(?:ghi)`, the pattern `(abc)` creates capturing group 1, `(def)` creates capturing group 2, and `(ghi)` is a group that is not captured.

Groups are used in conjunction with `str_match()` and `str_match_all()`.

Grouping and Capturing

Some examples of the common syntax for groups:

Pattern	Meaning
`a(bc)d`	Match the text `abcd`, capture the text in the group `bc`
`(?:abc)`	Non-capturing group
`(abc)def(ghi)`	Match `abcdefghi`, group `abc` and `ghi`
`(Mrs|Ms|Mr|Mx)`	`Mrs` or `Ms` or `Mr` or `Mx` (preference in the order given)
`\1`, `\2`, etc.	The first, second, etc. matched group (for `str_replace()`)

Note: Notice that the vertical line | is used for "or", just like in logical expressions. The vertical line | is called the **alternation** operator.

Grouping and Capturing Examples

The output of `str_match()` is a character matrix whose first column is the complete match, followed by one column for each capturing group.

```
str_match("abcdefghijkl", "(abc)(def)(?:ghi)")
```

```
##      [,1]         [,2]  [,3]
## [1,] "abcdefghi" "abc" "def"
```

Grouping and Capturing Examples

```
text <- "Mr. Wyatt, Mrs. Haverford, Ms. Knope"
pattern1 <- "Mrs|Ms|Mr|Mx"
str_match_all(text, pattern1)
```

```
## [[1]]
##      [,1]
## [1,] "Mr"
## [2,] "Mrs"
## [3,] "Ms"
```

```
pattern2 <- "Mr|Mrs|Ms|Mx"
```

Grouping and Capturing Examples

```
text <- "Mr. Wyatt, Mrs. Haverford, Ms. Knope, Andy Dwyer"
capture <- "(Mrs|Ms|Mr)\\. (\\w+)"
str_match_all(text,capture)
```

```
## [[1]]
##      [,1]             [,2]  [,3]
## [1,] "Mr. Wyatt"      "Mr"  "Wyatt"
## [2,] "Mrs. Haverford" "Mrs" "Haverford"
## [3,] "Ms. Knope"      "Ms"  "Knope"
```

```
non_capture <- "(?:Mrs|Ms|Mr)\\. (\\w+)"
str_match_all(text,non_capture)
```

```
## [[1]]
##      [,1]             [,2]
## [1,] "Mr. Wyatt"      "Wyatt"
## [2,] "Mrs. Haverford" "Haverford"
## [3,] "Ms. Knope"      "Knope"
```

Grouping and Capturing Examples

```
text <- "Leslie Knope, April Ludgate, Larry Gengurch"
pattern <- "(\\w+) (\\w+),"
str_match_all(text, pattern)
```

```
## [[1]]
##      [,1]              [,2]     [,3]
## [1,] "Leslie Knope,"   "Leslie" "Knope"
## [2,] "April Ludgate,"  "April"  "Ludgate"
```

```
str_replace_all(text, pattern, "\\2, \\1;")
```

```
## [1] "Knope, Leslie; Ludgate, April; Larry Gengurch"
```

Grouping and Capturing Examples

```
text <- "abcdefghijklmnopqrstuvwxyz"
pattern <- "b(\\w+)i"
str_match(text, pattern)
```

```
##      [,1]       [,2]
## [1,] "bcdefghi" "cdefgh"
```

```
pattern2 <- "a(\\w+)(e[a-h]+)"
str_match(text,pattern2)
```

```
##      [,1]       [,2]  [,3]
## [1,] "abcdefgh" "bcd" "efgh"
```

Telephone Numbers Example

For a more complicated example, we can define a regular expression to extract phone numbers.

```
text <- c("apple", "219 733 8965", "329-293-8753",
          "Work: 579-499-7527; Home: 543.355.3679")
phone_pattern <- "([2-9][0-9]{2})[- .]([2-9]{3}[- .][0-9]{4})"
str_extract_all(text, phone_pattern)
```

```
## [[1]]
## character(0)
##
## [[2]]
## [1] "219 733 8965"
##
## [[3]]
## [1] "329-293-8753"
##
## [[4]]
## [1] "579-499-7527" "543.355.3679"
```

Telephone Numbers Example

```
#text <- c("apple","219 733 8965","329-293-8753",
#           "Work: 579-499-7527; Home: 543.355.3679")
#phone_pattern <-
#  "([2-9][0-9]{2})[- .]([2-9]{3}[- .][0-9]{4})"
str_extract_all(text,phone_pattern)
```

```
## [[1]]
## character(0)
##
## [[2]]
## [1] "219 733 8965"
##
## [[3]]
## [1] "329-293-8753"
##
## [[4]]
## [1] "579-499-7527" "543.355.3679"
```

Telephone Numbers Example

```
#text <- c("apple","219 733 8965","329-293-8753",
#          "Work: 579-499-7527; Home: 543.355.3679")
#phone_pattern <-
#  "([2-9][0-9]{2})[- .]([2-9]{3}[- .][0-9]{4})"
str_match(text,phone_pattern)
```

```
##      [,1]             [,2]  [,3]
## [1,] NA               NA    NA
## [2,] "219 733 8965"   "219" "733 8965"
## [3,] "329-293-8753"   "329" "293-8753"
## [4,] "579-499-7527"   "579" "499-7527"
```

Telephone Numbers Example

```
#text <- c("apple","219 733 8965","329-293-8753",
#          "Work: 579-499-7527; Home: 543.355.3679")
#phone_pattern <-
#  "([2-9][0-9]{2})[- .]([2-9]{3}[- .][0-9]{4})"
str_match_all(text,phone_pattern)
```

```
## [[1]]
##      [,1] [,2] [,3]
##
## [[2]]
##      [,1]           [,2]  [,3]
## [1,] "219 733 8965" "219" "733 8965"
##
## [[3]]
##      [,1]           [,2]  [,3]
## [1,] "329-293-8753" "329" "293-8753"
##
## [[4]]
##      [,1]           [,2]  [,3]
## [1,] "579-499-7527" "579" "499-7527"
## [2,] "543.355.3679" "543" "355.3679"
```

Lookarounds

Occasionally we want to match characters that have a certain pattern before or after it. There are statements called **lookahead** and **lookbehind**, collectively called **lookarounds**, that look ahead or behind a pattern to check if a pattern does or does not exist.

Pattern	Name
`(?=...)`	Positive lookahead
`(?!...)`	Negative lookahead
`(?<=...)`	Positive lookbehind
`(?<!...)`	Negative lookbehind

The lookbehind patterns must have a bounded length (no * or +).

Positive Lookahead

Positive lookahead with `(?=...)` looks ahead of the current match to ensure that the `...` subpattern matches.

```
text <- "I put a grey hat on my grey greyhound."
pattern <- "grey(?=hound)"
str_extract_all(text, pattern)
```

```
## [[1]]
## [1] "grey"
```

The word `grey` is matched *only* if it is followed by `hound`. Note that `hound` itself is not part of the match.

Negative Lookahead

Negative lookahead with `(?!...)` looks ahead of the current match to ensure that the `...` subpattern does *not* match.

```
text <- "I put a grey hat on my grey greyhound."
pattern <- "grey(?!hound)"
str_extract_all(text, pattern)
```

```
## [[1]]
## [1] "grey" "grey"
```

The word `grey` is matched *only* if it is *not* followed by `hound`.

Positive Lookbehind

Positive lookbehind with `(?<=...)` looks behind the current position to ensure that the `...` subpattern immediately precedes the current match.

```
text <-
  "I withdrew 100 $1 bills, 20 $5 bills, and 5 $20 bills."
pattern <- "(?<=\\$)[[:digit:]]+"
str_extract_all(text, pattern)
```

```
## [[1]]
## [1] "1"  "5"  "20"
```

The digits are matched *only* if they are immediately preceded by a dollar $ sign.

Negative Lookbehind

Positive lookbehind with `(?<!...)` looks behind the current position to ensure that the `...` subpattern does *not* immediately precede the current match.

```
text <- "I withdrew 100 $1 bills, 20 $5 bills, and 5 $20 bills."
pattern <- "(?<!\\$)[[:digit:]]+"
str_extract_all(text, pattern)
```

```
## [[1]]
## [1] "100" "20"  "5"   "0"
```

The digits are matched *only* if they are *not* immediately preceded by a dollar $ sign.

Question: Why does the 0 also match? How can you change the pattern to only return the numbers of bills?